



河南大學
HENAN UNIVERSITY

计算机操作系统

主讲人：胡 萍

明德，新民

止于至善



第五章 虚拟存储器

第1讲



本章内容

5.1 虚拟存储器概述

5.2 请求分页存储管理方式

5.3 页面置换算法

5.4 “抖动”与工作集

5.5 请求分段存储管理方式





5.1 虚拟存储器概述

- 5.1.1 虚拟存储器的引入
- 5.1.2 虚拟存储器的定义和特征
- 5.1.3 虚拟存储器的实现方法



5.1 虚拟存储器概述

- 前面介绍的各种存储管理方式，都需要将作业**全部装入**内存后方可运行。于是，出现了下面两种情况：
 - 当作业要求的内存空间超过内存总容量时，无法装入内存运行；
 - 有大量作业要求运行，但内存容量不足以容纳所有这些作业，只能将少量作业装入内存运行。
- 解决内存容量不够问题的办法：
 - 从物理上扩充内存；
 - 从**逻辑上**扩充内存——**虚拟存储器**。



5.1.1 虚拟存储器的引入

1. 常规存储器管理方式的特征

- 一次性
 - 驻留性
- 使不用或暂不用的程序（数据）占据了大量内存空间，使得需要运行的作业无法装入运行。

2. 局部性原理

- 程序执行呈现局部性规律，即在一较短的时间内，程序的执行仅局限于某个部分；相应地，它所访问的存储空间也局限于某个区域。
- 时间局部性：某指令一旦执行，则不久后该指令可能再次被执行（数据亦然）。循环
- 空间局部性：程序一旦访问了某个存储单元，不久后，附近的存储单元也将被访问。顺序执行



5.1.1 虚拟存储器的引入

3. 虚拟存储器基本原理

部分
装入

- 应用程序在运行前，仅将那些当前要运行的页面或段先装入内存便可以运行，其余部分暂留在磁盘上。

请求
调页

- 程序在运行时，如果所要访问的页（段）已调入内存，便可继续执行下去；否则，应利用OS所提供的请求调页（段）功能，将它们调入内存，以便继续运行。

页面
置换

- 如果此时内存已满，则需利用页（段）置换功能，将内存中暂不用的页（段）调到磁盘上在将访问的页（段）调入内存，使程序继续运行下去。



5.1.2 虚拟存储器的定义和特征

1. 虚拟存储器的定义

所谓虚拟存储器，是指具有**请求调入功能和置换功能**，能从**逻辑上对内存容量加以扩充**的一种存储系统。

2. 虚拟存储器的特征

- **多次性**：一个作业被分成多次调入内存运行(部分装入)；
- **对换性**：作业在运行过程中进行换进、换出；
- **虚拟性**：从逻辑上扩充内存，使用户看到的内存容量大于实际内存容量。这是虚拟存储器所表现的最重要的特征，也是实现虚拟存储器的最重要的目标。



5.1.3 虚拟存储器的实现方法

1. 请求分页系统

□ 在**分页系统基础**上，增加了**请求调页功能和页面置换功能**所形成的页式虚拟存储系统。

将缺失页面从外存调入内存

□ 必须提供硬件支持和相应的软件

将暂时用不到的页面从内存换出到外存

■ 硬件支持

- 请求分页的页表机制
- 缺页中断机构
- 地址变换机构

■ 实现请求分页的软件——请求调页、实现页面置换的软件



5.1.3 虚拟存储器的实现方法

2. 请求分段系统

- ❑ 在分段系统基础上，增加了请求调段功能和分段置换功能所形成的段式虚拟存储系统。
- ❑ 硬件支持
 - 请求分段的段表机制
 - 缺段中断机构
 - 地址变换机构
- ❑ 实现请求分段的软件——请求调段、实现段置换的软件

3. 请求段页系统



总结

不管哪种实现方式，都需要有一定的硬件支持，一般包括：

- 一定容量的**内存和外存**；
- **页（段）表机制**：作为主要的数据结构；
- **缺页中断机构**：当用户程序要访问的部分尚未调入内存时，
则产生中断；
- **地址变换机构**：实现逻辑地址到物理地址的转换。



5.2 请求分页存储管理方式

- 5.2.1 请求分页中的硬件支持
- 5.2.2 请求分页中的内存分配
- 5.2.3 页面调入策略



5.2.1 请求分页中的硬件支持

1. 页表机制

每个页表表项如下所示：

页号	物理块号	状态位P	访问位A	修改位M	外存地址
----	------	------	------	------	------

- **状态位P**：指示该页是否已调入内存，又称驻留位。
- **访问位A**：记录本页在一段时间内被访问的次数。
- **修改位M**：表示该页被调入内存后是否被修改过。
- **外存地址**：指出该页在外存的地址，通常是磁盘物理块号。

页号	物理块号	状态位P	访问位A	修改位M	外存地址
0	a	1	0	0	A
1	无	0	0	0	B



5.2.1 请求分页中的硬件支持

2. 缺页中断机构

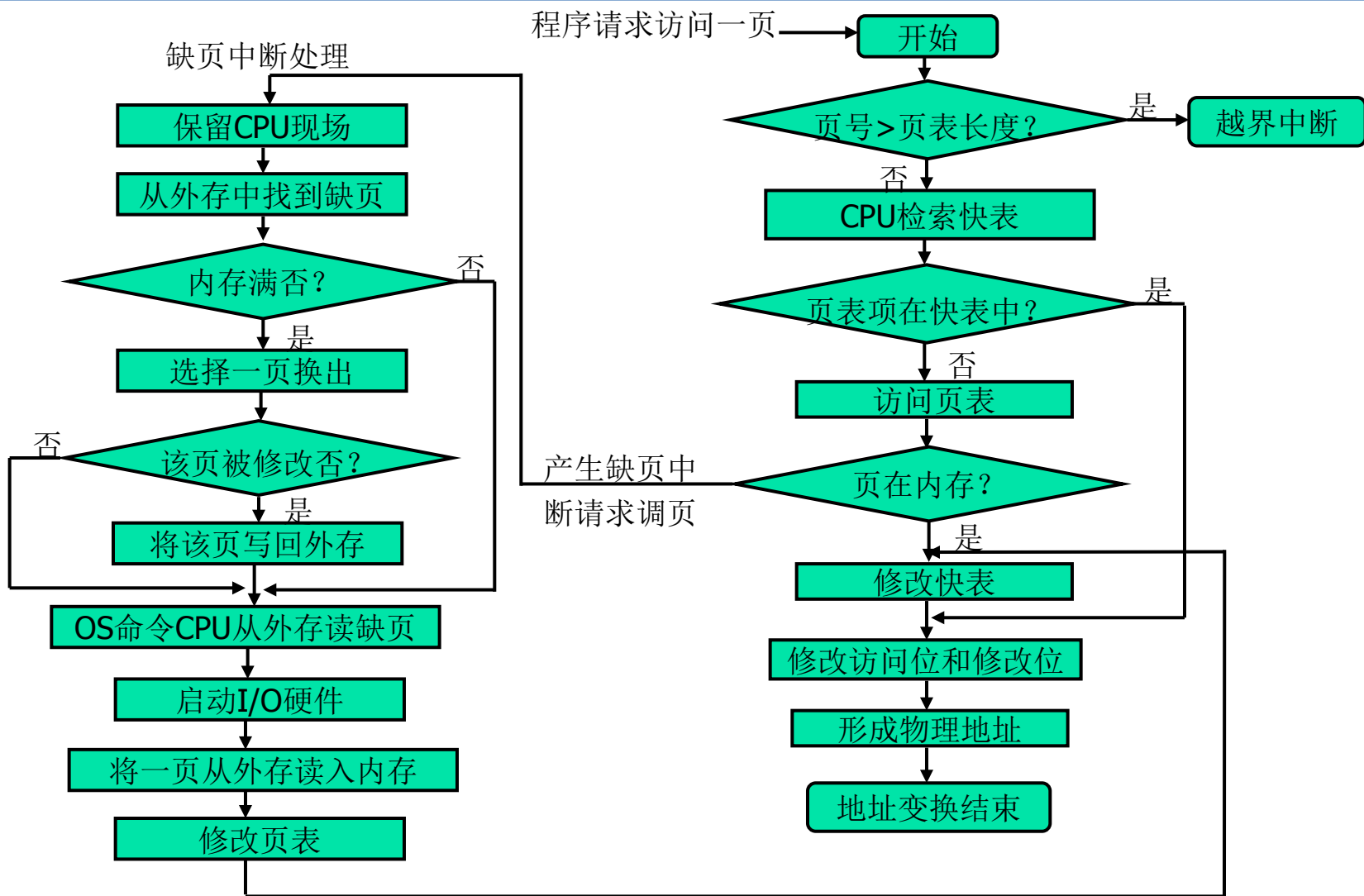
请求分页系统中，每当要访问的页面不在内存时，便产生一个**缺页中断**，**请求缺页中断处理程序处理**。此时缺页进程阻塞，放入阻塞队列。去外存将所缺页面调入内存空物理块，并修改相应页表项；若内存无空块，则由页面置换算法选择一个页面淘汰，若被淘汰页面有修改，还要将其写回外存。

缺页中断与一般中断相比，有以下两个明显区别：

- 缺页中断在**指令执行期间**产生和处理；
- 一条指令执行期间可能要产生**多次**缺页中断处理。



3. 地址变换机构





3. 地址变换机构

地址变换流程如下：

- ① 先检索快表，若命中，从相应表项中取出页面对应物理块号，并修改页表项中的访问位；对于写指令，还需将修改位置1；
- ② 若快表未命中，查找页表，若找到，从相应页表项中取出物理块号，并将该页表项写入快表，若快表已满，则采用替换算法替换；
- ③ 若页表中未找到，进行缺页中断处理，请求系统将所缺页面从外存换入内存。若内存有空物理块，调入所缺页面后，由操作系统更新页表和快表，并修改相应页表项。
- ④ 若内存已满，采用页面置换算法进行页面置换，若被淘汰页面有修改，还要将其写回外存。调入所缺页面后，由操作系统更新页表和快表，并修改相应页表项。
- ⑤ 最后形成物理地址，并访存。



5.2.2 请求分页中的内存分配

为进程分配内存时，涉及到3个问题：

1. 最小物理块数的确定（驻留集大小）

- 取决于指令的格式、功能和寻址方式
- 物理块数越少，驻留在内存的进程越多，可提高多道程序的并发度；但进程缺页率较高，CPU需耗费大量时间处理缺页。
- 物理块数越多，导致多道程序并发度下降；同时，当分给某进程块数达到一定数目时，再增加块数对缺页率的改善并不明显，反而浪费内存空间。



5.2.2 请求分页中的内存分配

2. 内存分配策略

□ 两种内存分配策略

- **固定分配**：为每个进程分配固定数目的物理块，在进程运行期间不再改变。
- **可变分配**：先为每个进程分配一定数目的物理块，在进程运行期间，可根据情况做适当的增加或减少。

□ 两种置换策略

- **全局置换**：若进程在运行中发现缺页，则将OS所保留的空闲物理块取出一块分配给进程，或从所有物理块中选出一块换出，然后将所缺页面换入。
- **局部置换**：若进程在运行中发现缺页，只能从分配给该进程的页面中选出一块进行置换，以保证分配空间不变。



5.2.2 请求分页中的内存分配

2. 内存分配策略

1) 固定分配局部置换

实现时，难以确定应为进程分配的物理块数。

2) 可变分配全局置换

最易于实现的物理块分配和置换策略，已用于若干OS中。但是会盲目增加进程物理块数，导致系统多道程序并发度下降。

3) 可变分配局部置换

基于进程类型或程序员需求，为进程分配一定数目的物理块。当发生缺页时，只允许从该进程在内存的页面中进行置换，不会影响其他进程。若频繁缺页，系统再为进程追加物理块数，反之减少物理块数，直至缺页率趋于适当值。这种方法保证进程低缺页率同时，也保持系统多道程序并发度。但是实现复杂，开销大。



5.2.2 请求分页中的内存分配

3. 物理块分配算法

- **平均分配算法**：将系统重可供分配的物理块平均分配给个进程。**显然不太合理**
- **按比例分配算法**：根据进程的大小按比例分配物理块。
- **考虑优先权的分配算法**：为重要的、紧迫的进程分配更多的物理块。通常采取的方法是将所有可分配的物理块分成两部分：一部分按比例；另一部分根据各进程的优先权。这种分配算法更为合理。



5.2.3 页面调入策略

1. 何时调入页面

- 预调页策略：主要用于进程的首次调入，由程序员指定；
- 请求调页策略：进程运行期间，有系统指定。目前虚拟存储器中大多采用此策略。

2. 从何处调入页面

- 系统有足够的对换区：全部从对换区调入
- 系统缺少足够对换区：不会被修改的从文件区调入；可能被修改的从对换区调入。
- UNIX方式：未运行的页面，从文件区调入；曾经运行过又被调出的页面，从对换区调入。共享页面有时无须调入。



5.2.3 页面调入策略

3. 页面调入过程

当进程要访问的页面不在内存时（状态位为0），便向CPU发出缺页中断，中断响应后转入缺页中断处理程序。首先查找页表得到该页的物理块，若内存未满，则启动磁盘I/O，将所缺页调入内存，并修改页表。若内存已满，按某种页面置换算法从内存中置换一个页面，若被置换页面被修改过（修改位为1），则必须将该页面写回磁盘，所缺页面换入内存后，修改页表中的相应表项，置其状态位为1。

整个页面的调入过程对用户是透明的。



5.2.3 页面调入策略

4. 缺页率

进程运行过程中，访问页面成功次数为S，缺失次数为F，则该进程运行过程中的缺页率为：

$$f = \frac{F}{S+F}$$



5.3 页面置换算法

- 5.3.1 最佳置换算法和先进先出置换算法
- 5.3.2 最近最久未使用（LRU）置换算法
- 5.3.3 Clock置换算法

当缺页中断发生，需要调入新的页面而内存已满时，
利用页面置换算法进行页面置换。

好的页面置换算法，应具有较低的页面更换频率

【例5-1】假定系统为某进程分配3个空物理块，考虑引用以下20个页面：7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1。

5.3.1 最佳置换算法和先进先出置换算法

1. 最佳置换算法（OPT） 性能最好，但无法实现

- 一种理论上的算法：最低缺页率
- 淘汰以后永不使用或是未来最长时间不再使用的页面。

解：

T		1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
引用页面		7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1
物理块	0	7	7	7	2	2	2	2	2	2	2	2	2	2	2	2	2	2	7	7	7
	1		0	0	0	0	0	0	4	4	4	0	0	0	0	0	0	0	0	0	0
	2			1	1	1	3	3	3	3	3	3	3	3	1	1	1	1	1	1	1
是否缺页		√	√	√	√		√		√			√			√				√		
被置换页				7			1	0			4			3				2			

图5-3 利用最佳页面置换算法的置换图

发生了9次缺页中断，缺页率=9/20；产生6次页面置换

【例5-1】假定系统为某进程分配3个空物理块，考虑引用以下20个页面：7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1。

5.3.1 最佳置换算法和先进先出置换算法

2. 先进先出（FIFO）页面置换算法 实现简单，但性能差

- 淘汰在内存中驻留时间最长（即最早进入内存）的页面。
- 实现方法：将已调入内存的页面按先后次序链接成一个队列，置换时选择置换指针指向的最老的队首页面。

解：

T	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
引用页面	7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1
物理块	0	→7	→7	→7	2	2	→2	4	4	→4	0	0	0	0	→0	→0	→0	7	7	→7
	1		0	0	→0	→0	3	3	→3	2	→2	→2	→2	1	1	1	1	→1	0	0
	2			1	1	1	→1	0	0	→0	3	3	3	→3	2	2	2	2	→2	1
是否缺页	√	√	√	√		√	√	√	√	√	√			√	√			√	√	√
被置换页				7		0	1	2	3	0	4			2	3			0	1	2

图5-4 利用FIFO页面置换算法的置换图

发生了15次缺页中断，缺页率=15/20；产生12次页面置换

【例5-1+】若系统为某进程分别分配3个和4个空物理。考虑引用以下12个页面：3, 2, 1, 0, 3, 2, 4, 3, 2, 1, 0, 4。求缺页次数。

2. 先进先出（FIFO）页面置换算法

解：

3个
9次
缺页

T		1	2	3	4	5	6	7	8	9	10	11	12
引用页面		3	2	1	0	3	2	4	3	2	1	0	4
物理块	0	→3	→3	→3	0	0	→0	4	4	4	4	→4	→4
	1		2	2	→2	3	→3	→3	→3	→3	1	1	1
	2			1	1	→1	2	2	2	2	→2	0	0
是否缺页		√	√	√	√	√	√				√	√	
被置换页					3	2	1	0			3	2	

4个
10次
缺页

T		1	2	3	4	5	6	7	8	9	10	11	12
引用页面		3	2	1	0	3	2	4	3	2	1	0	4
物理块	0	→3	→3	→3	→3	→3	→3	4	4	4	→4	0	0
	1		2	2	2	2	2	→2	3	3	3	→3	4
	2			1	1	1	1	1	→1	2	2	2	→2
	3				0	0	0	0	0	→0	1	1	1
是否缺页		√	√	√	√			√	√	√	√	√	√
被置换页								3	2	1	0	4	3

FIFO算法中出现了为进程分配更多物理块，缺页次数不减反增的异常现象，称为 **Belady异常**。

【例5-1】假定系统为某进程分配3个物理块，考虑引用以下20个页面：7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1。

5.3.2 最近最久未使用（LRU）置换算法

- 选择最近最久未使用的页面进行淘汰。

性能好，但需要专门硬件支持，算法开销大。

解：

T	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
引用页面	7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1
物理块	0	7	7	7	2	2	2	4	4	4	0	0	0	1	1	1	1	1	1	1
	1		0	0	0	0	0	0	0	3	3	3	3	3	3	0	0	0	0	0
	2			1	1	1	3	3	3	2	2	2	2	2	2	2	2	7	7	7
是否缺页	√	√	√	√		√		√	√	√	√			√		√		√		
被置换页				7		1		2	3	0	4			0		3		2		
上次访问时刻	7:1				2:4		2:4 4:8 4:8 4:8				0:11				1:14		1:17			
	0:2				0:5		0:7 0:7 0:7 3:10				3:12				3:12		0:16			
	1:3				1:3		3:6 3:6 2:9 2:9				2:13				2:13		2:15			

图5-5 利用LRU页面置换算法的置换图

发生了12次缺页中断，缺页率=12/20；产生9次页面置换

【例5-1】假定系统为某进程分配3个空物理块，考虑引用以下20个页面：7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1。

5.3.2 最近最久未使用（LRU）置换算法

- 选择最近最久未使用的页面进行淘汰。
- 实现方法2：用栈保存当前使用的页面号，栈顶始终是最新的，栈底是最近最久未使用的。置换时，置换选择栈底页面。

性能好，但需要专门硬件支持，算法开销大。

解：

T	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
引用页面	7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1
物理块	0	7	7	7	2	2	2	4	4	4	0	0	0	1	1	1	1	1	1	1
	1		0	0	0	0	0	0	0	3	3	3	3	3	3	0	0	0	0	0
	2			1	1	1	3	3	3	2	2	2	2	2	2	2	2	7	7	7
是否缺页	√	√	√	√		√		√	√	√	√			√		√		√		
被置换页				7		1		2	3	0	4			0		3		2		
栈	7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1
		7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0
			7	0	1	2	2	3	0	4	2	2	0	3	3	1	2	0	1	7

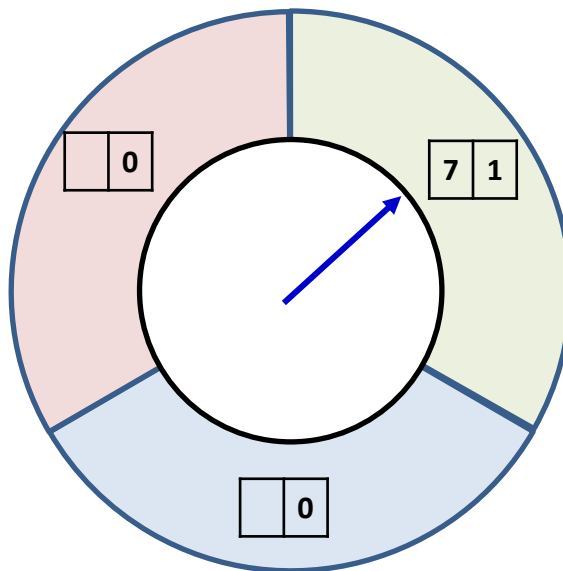
发生了12次缺页中断，缺页率=12/20；产生9次页面置换



页号

访问位

被访问时置1
未被访问置0

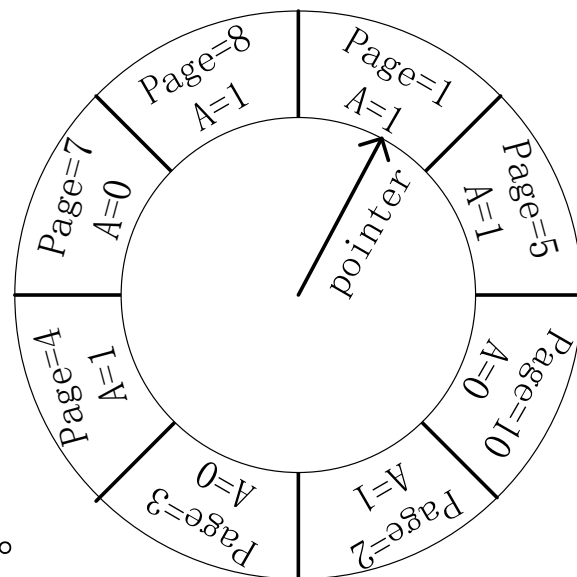




5.3.3 Clock置换算法 性能和开销较为均衡的算法

1. 简单Clock置换算法 最多经过两轮扫描

- 内存中所有页面组织在一个循环链表中，并设置一个置换指针pointer，用于指示页面查找的起始位置。
- 设置一个访问位A，当页面被访问时，访问位置1。
- 置换页面时，从置换指针位置开始，查看页面访问位A，若为1，将该页面访问位修改为0，暂不换出，继续检查下一个页面。若为0，将该页面换出内存，并将新页面换入内存，置新页面访问位为1。最后让指针指向下一个页面。
- 第1轮扫描结束后，若没找到淘汰的页面，则进行第2轮扫描，必定能找到淘汰的页面。



【例5-1】假定系统为某进程分配3个空物理块，考虑引用以下20个页面：7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1。

5.3.3 Clock置换算法

1. 简单Clock置换算法

解： 设：蓝色为置换指针位置，*为访问标志为1

T		1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
引用页面		7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1
物理块	0	7*	7*	7*	2*	2*	2*	2*	4*	4*	4*	4	3*	3*	3	3	0*	0*	0	0*	0*
	1		0*	0*	0	0*	0	0*	0	2*	2*	2	2	2*	1*	1*	1*	1*	7*	7*	7*
	2			1*	1	1	3*	3*	3	3	3*	0*	0*	0*	0	2*	2*	2*	2	2	1*
是否缺页		√	√	√	√		√		√	√		√	√		√	√	√		√		√
被置换页					7		1		2	0		3	4		2	0	3		1		2

发生了14次缺页中断，缺页率=14/20；产生**11**次页面置换



5.3.3 Clock置换算法

2. 改进型Clock置换算法(UNIX SVR4采用此算法)

淘汰被修改过的页面时，需将其写回磁盘(置换代价高)，因此应淘汰既未被访问又未被修改的页面。为此，每个页面除了有访问位A外，还增加一个修改位M。由访问位A与修改位M可以组成下面4种类型的页面：

- 1类(A=0,M=0)，是最佳淘汰页；
- 2类(A=0,M=1)；
- 3类(A=1,M=0)；
- 4类(A=1,M=1)，最近被访问且被修改过的页，最不应该淘汰。



5.3.3 Clock置換算法

2. 改进型Clock置換算法(UNIX SVR4采用此算法)

- ① 从指针当前位置开始，扫描循环队列，寻找 $A=0$ 且 $M=0$ 的第1类页面，将所遇到的第一个页面淘汰。
- ② 若第①步查找一轮后未遇到第1类页面，则寻找 $A=0$ 且 $M=1$ 的第2类页面，将所遇到的第一个页面淘汰。第2轮扫描期间将所有扫描过的页面访问位 A 清0。
- ③ 若第②步扫描失败，则将指针返回开始位置，同时将所有的访问位清0，然后重复①；若仍失败，再重复第②步，此时就一定能找到被淘汰的页。

淘汰原3类页

淘汰原4类页

1. 有一个页式虚存系统，某进程占用3个内存块，开始时内存为空，执行如下访问页号序列：

0, 1, 2, 3, 4, 1, 2, 0, 5, 1, 0, 1, 2, 3, 2, 4, 5

(1) 采用先进先出（**FIFO**）置换算法，缺页次数是多少？

(2) 采用**LRU**置换算法，缺页次数是多少？

(3) 若用最佳置换（**OPT**）算法呢？

2. 在一个请求分页系统中，采用**LRU**页面置换算法时，假如一个作业的页面走向为1、3、2、1、1、3、5、1、3、2、1、5，当分配给该作业的物理块数**M**分别为3和4时，试计算在访问过程中所发生的缺页次数和缺页率，并比较所得结果。



练习题



1. 有一个页式虚存系统，某进程占用3个内存块，开始时内存为空，执行如下访问页号序列：

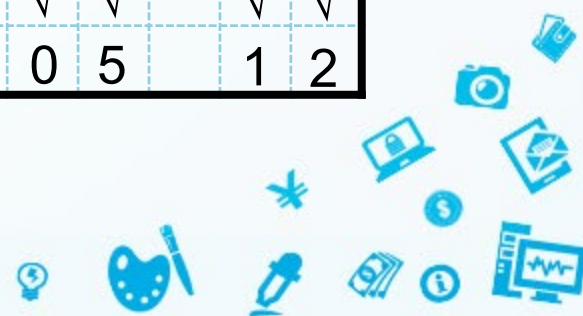
0, 1, 2, 3, 4, 1, 2, 0, 5, 1, 0, 1, 2, 3, 2, 4, 5

(1) 采用先进先出 (FIFO) 置换算法，缺页次数是多少？

14

解：

T		1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
引用页面		0	1	2	3	4	1	2	0	5	1	0	1	2	3	2	4	5
物理块	0	→0	→0	→0	3	3	→3	2	2	→2	1	1	1	1	→1	→1	4	4
	1		1	1	→1	4	4	→4	0	0	→0	→0	→0	2	2	2	→2	5
	2			2	2	→2	1	1	→1	5	5	5	5	→5	3	3	3	→3
是否缺页		√	√	√	√	√	√	√	√	√	√			√	√		√	√
被置换页					0	1	2	3	4	1	2			0	5		1	2



练习题



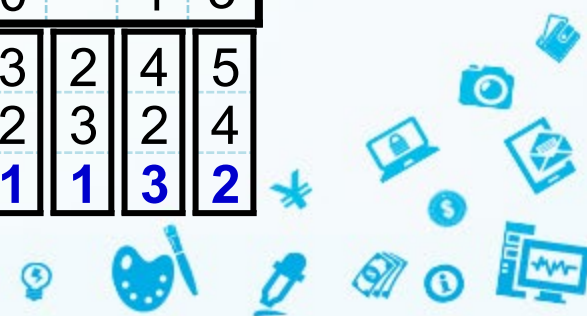
1. 有一个页式虚存系统，某进程占用3个内存块，开始时内存为空，执行如下访问页号序列：

0, 1, 2, 3, 4, 1, 2, 0, 5, 1, 0, 1, 2, 3, 2, 4, 5

(2) 采用LRU置换算法，缺页次数是多少？ **14**

解：

T		1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
引用页面		0	1	2	3	4	1	2	0	5	1	0	1	2	3	2	4	5
物理块	0	0	0	0	3	3	3	2	2	2	1	1	1	1	1	1	4	4
	1		1	1	1	4	4	4	0	0	0	0	0	0	3	3	3	5
	2			2	2	2	1	1	1	5	5	5	5	2	2	2	2	2
是否缺页		√	√	√	√	√	√	√	√	√	√			√	√		√	√
被置换页					0	1	2	3	4	1	2			5	0		1	3
顶		0	1	2	3	4	1	2	0	5	1	0	1	2	3	2	4	5
栈			0	1	2	3	4	1	2	0	5	1	0	1	2	3	2	4
				0	1	2	3	4	1	2	0	5	5	0	1	1	3	2



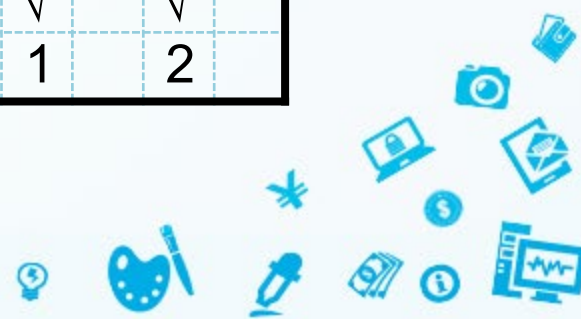
1. 有一个页式虚存系统，某进程占用3个内存块，开始时内存为空，执行如下访问页号序列：

0, 1, 2, 3, 4, 1, 2, 0, 5, 1, 0, 1, 2, 3, 2, 4, 5

(3) 若用最佳置换（OPT）算法呢？10

解：

T		1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
引用页面		0	1	2	3	4	1	2	0	5	1	0	1	2	3	2	4	5
物理块	0	0	0	0	3	4	4	2	2	5	5	5	5	5	5	5	5	5
	1		1	1	0	0	0	0	0	0	0	0	0	2	2	2	4	4
	2			2	1	1	1	1	1	1	1	1	1	1	3	3	3	3
是否缺页		√	√	√	√	√		√		√				√	√		√	
被置换页					0	3		4		2				0	1		2	



练习题



2. 在一个请求分页系统中，采用LRU页面置换算法时，假如一个作业的页面走向为1、3、2、1、1、3、5、1、3、2、1、5，当分配给该作业的物理块数M分别为3和4时，试计算在访问过程中所发生的缺页次数和缺页率，并比较所得结果。

6 6/12

解：

T		1	2	3	4	5	6	7	8	9	10	11	12
引用页面		1	3	2	1	1	3	5	1	3	2	1	5
物理块	0	1	1	1	1	1	1	1	1	1	1	1	1
	1		3	3	3	3	3	3	3	3	3	3	5
	2			2	2	2	2	5	5	5	2	2	2
是否缺页		√	√	√				√			√		√
被置换页								2			5		3

顶		1	3	2	1	1	3	5	1	3	2	1	5
栈			1	3	2	2	1	3	5	1	3	2	1
				1	3	3	2	1	3	5	1	3	2

练习题



河南大学
Henan University

2. 在一个请求分页系统中，采用LRU页面置换算法时，假如一个作业的页面走向为1、3、2、1、1、3、5、1、3、2、1、5，当分配给该作业的物理块数M分别为3和4时，试计算在访问过程中所发生的缺页次数和缺页率，并比较所得结果。

解：

T	1	2	3	4	5	6	7	8	9	10	11	12
引用页面	1	3	2	1	1	3	5	1	3	2	1	5
物理块	0	1	1	1	1	1	1	1	1	1	1	1
	1		3	3	3	3	3	3	3	3	3	5
	2			2	2	2	2	2	2	2	2	2
	3						5	5	5	5	5	5
是否缺页	√	√	√				√					
被置换页												

栈	1	3	2	1	1	3	5	1	3	2	1	5
		1	3	2	2	1	3	5	1	3	2	1
			1	3	3	2	1	3	5	1	3	2
							2	2	2	5	5	3





5.5 请求分段存储管理方式

□ 5.5.1 请求分段中的硬件支持

□ ~~5.5.2 分段的共享和保护~~



5.5 请求分段存储管理方式

- ❑ 在请求分段系统中，程序运行之前，只需调入若干分段（不必调入所有分段），便可启动运行。
- ❑ 当所访问的段不在内存时，可请求OS将所缺的段调入内存。
- ❑ 像请求分页系统一样，为实现请求分段存储管理功能，同样需要一定的硬件支持和相应的软件。



5.5.1 请求分段中的硬件支持

1. 请求段表机制

请求分段式管理中，所需的最主要数据结构是段表。

段名	段长	段基址	存取 方式	访问字段 A	修改位 M	存在位 P	增补位	外存 始址
----	----	-----	----------	-----------	----------	----------	-----	----------

- **存取方式**：用于标识本分段的存取属性是执行、只读、读/写
- **访问字段A**：用于记录该分段被访问的频繁程度
- **修改位M**：用于表示该段进入内存后是否被修改，供分段置换时参考
- **存在位P**：指示本段是否已调入内存，供程序访问时参考
- **增补位**：用于表示该段进入内存后是否做过动态增长
- **外存始址**：指示本段在外存中的起始地址，即起始盘块号



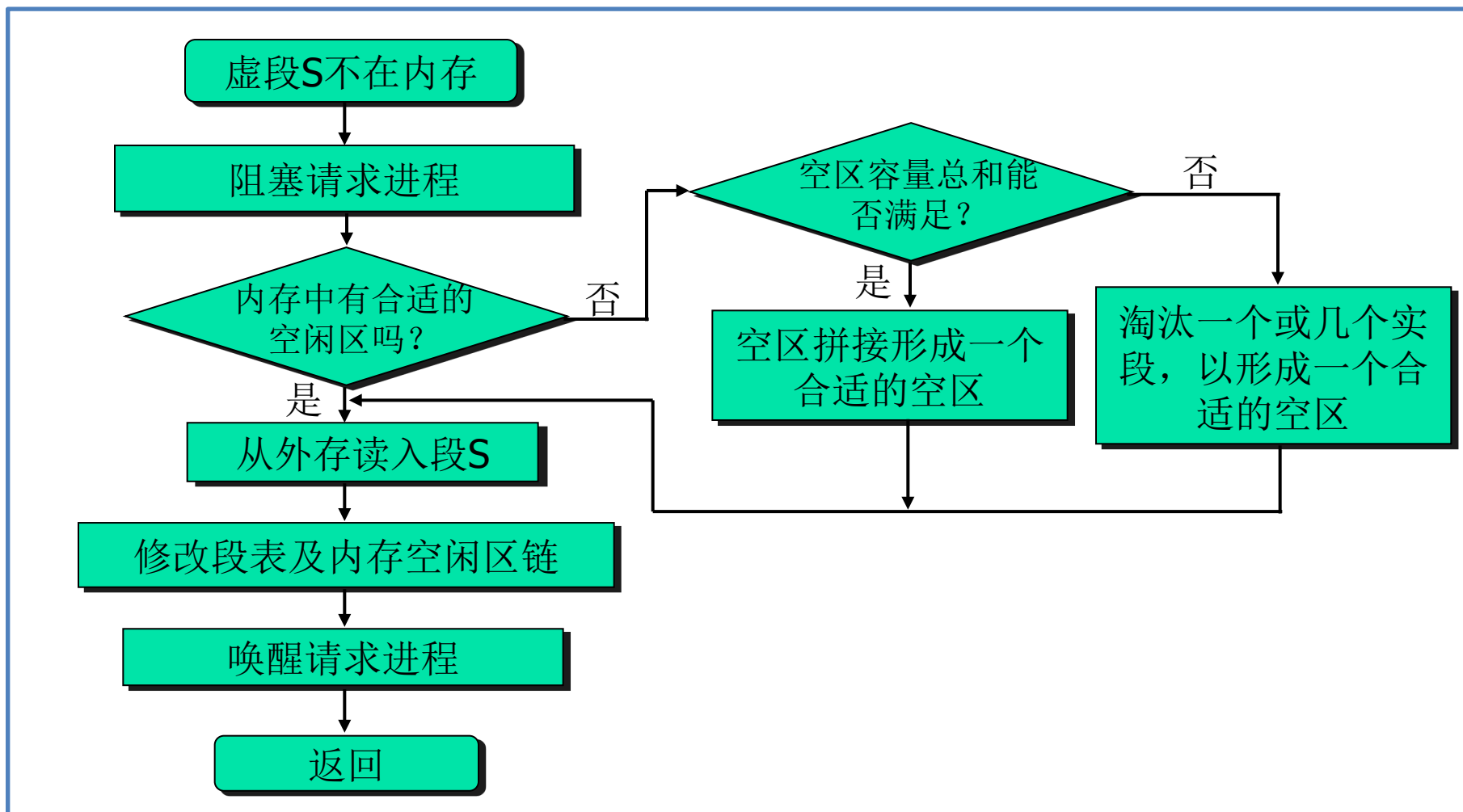
5.5.1 请求分段中的硬件支持

2. 缺段中断机构

- 在请求分段系统中，每当发现运行进程所要访问的段不在内存时，便由缺段中断机构产生一缺段中断信号，进入OS后由缺段中断处理程序将所需的段调入内存。
- 同样需要在一条指令的执行期间，产生和处理中断（一条指令执行期间可能产生多次缺段中断）
- 缺段中断的处理比缺页中断复杂（段不是定长）



缺段中断的处理过程





5.5.1 请求分段中的硬件支持

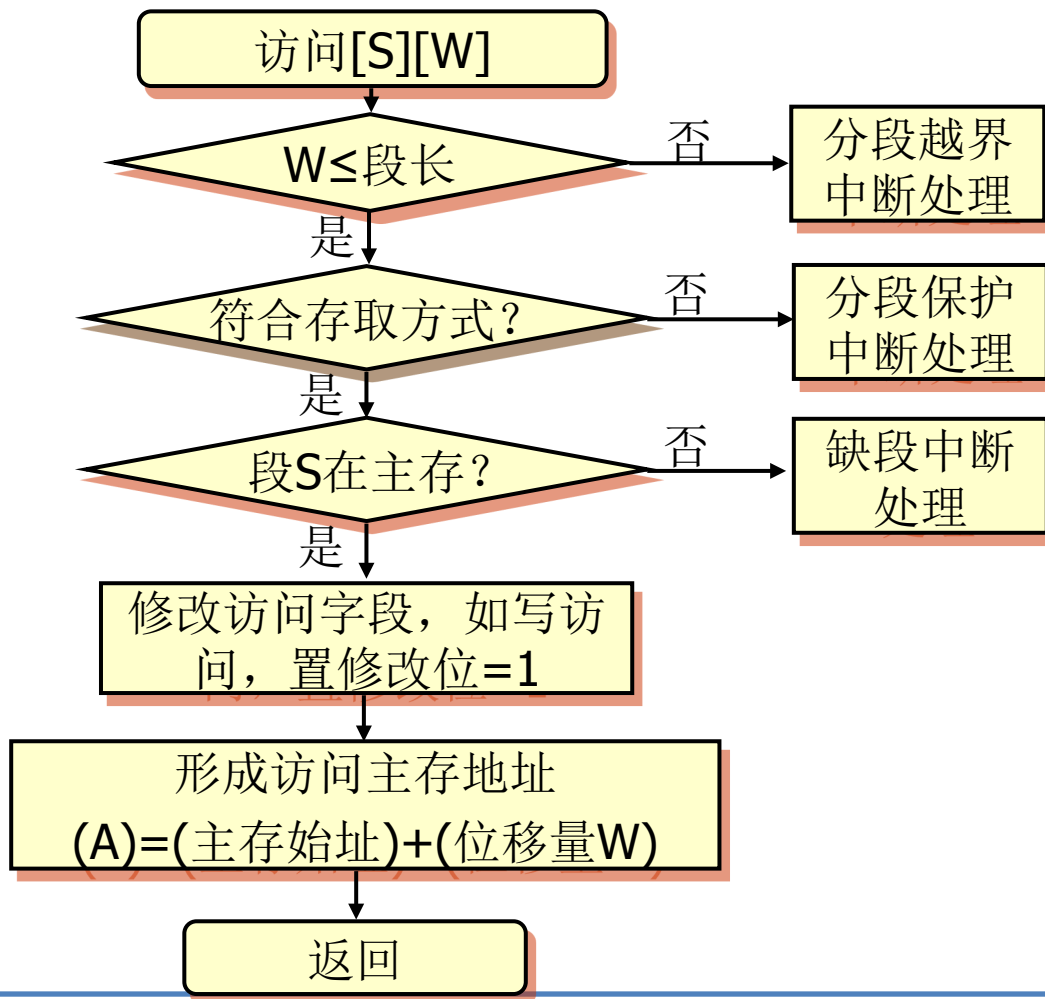
3. 地址变换机构

请求分段系统的地址变换机构，是在分段系统的地址变换机构的基础上形成的。

因为在地址变换时，若发现所要访问的段不在内存，必须先将所缺的段调入内存，并修改段表，然后才能利用段表进行地址变换。因此在地址变换机构中增加了缺页中断的请求和处理等功能。



请求分段系统的地址变换过程





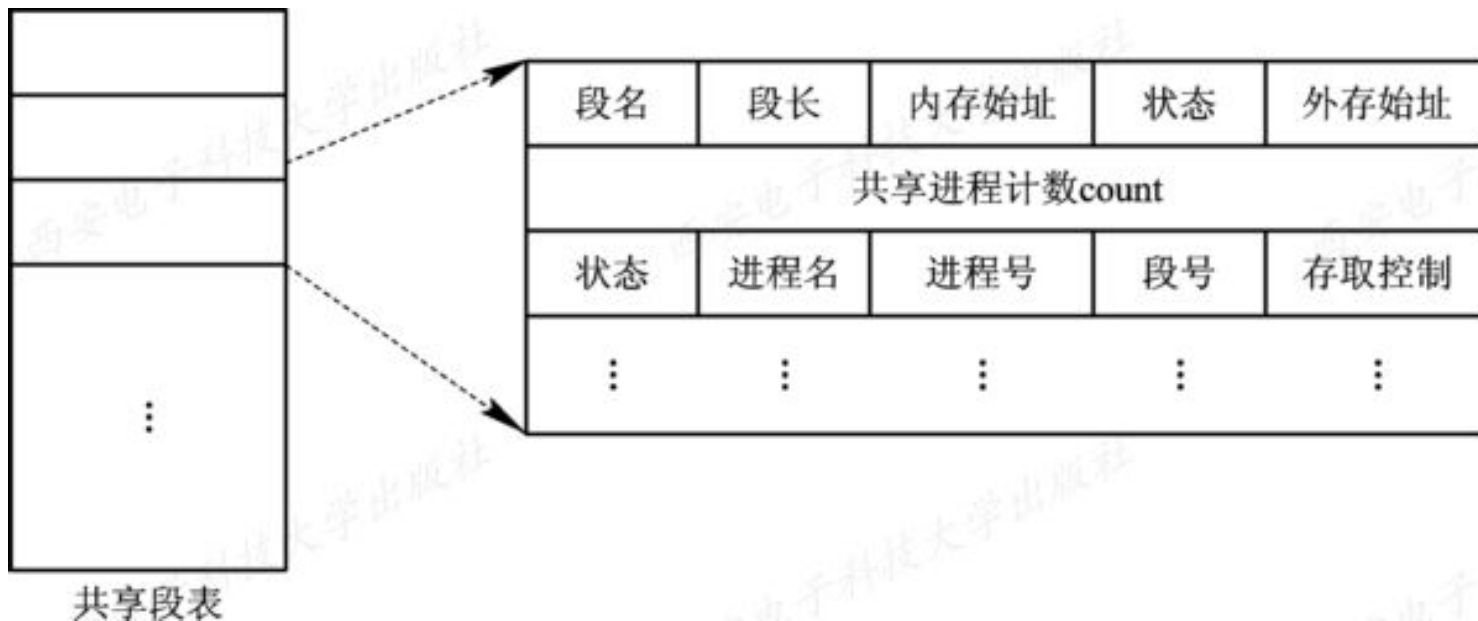
5.5.2 分段的共享和保护（自学）

1. 共享段表

(1) 共享进程计数count

(2) 存取控制字段

(3) 段号

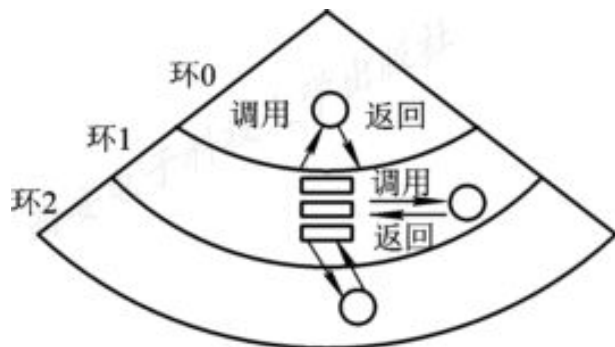




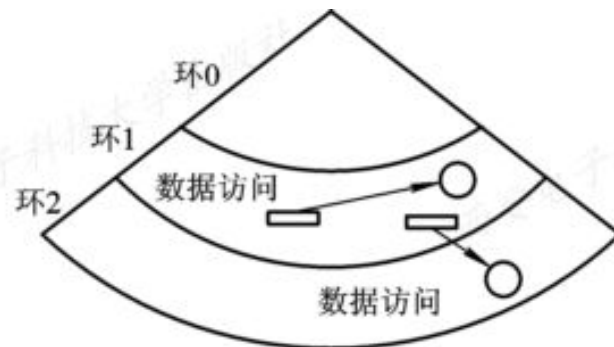
5.5.2 分段的共享和保护

2. 共享段的分配与回收

(1) 共享段的分配



(a) 程序间的控制传输



(b) 数据访问

目前，常采用以下几种措施来确保信息的安全。

- 1) 越界检查
- 2) 存取控制检查
- 3) 环保护机构



河南大學
HENAN UNIVERSITY

明德，新民
止于至善



The End ...

